Chapter 5: Advanced SQL

Chapter 5: Advanced SQL

- **Accessing SQL From a Programming Language**
  - Dynamic SQL
    - JDBC and ODBC
  - Embedded SQL
- **Functions and Procedural Constructs**
- **Triggers**

JDBC and ODBC

- API (application-program interface) for a program to ***interact with a database server***
- Application makes calls to
  - ***Connect with the database server***
  - ***Send SQL commands to the database server***
  - ***Fetch tuples of result one-by-one into program variables***
- **ODBC** (Open Database Connectivity) works with C, C++, C#, and Visual Basic
  - Other API's such as ADO.NET sit on top of ODBC
- **JDBC** (Java Database Connectivity) works with Java

JDBC

- **JDBC** is a Java API for communicating with database systems supporting SQL.
- JDBC supports ***querying and updating data***, and ***retrieving query results***.
- JDBC also supports ***metadata retrieval*** (relations present in the database and the names and types of relation attributes)
- **Model for communicating with the database**:
  - ***Open a connection***
  - ***Create a "statement" object***
  - ***Execute queries using the Statement object to send queries and fetch results***
  - ***Exception mechanism to handle errors***

ODBC

- **Open DataBase Connectivity (ODBC)** standard
  - standard for application program to ***communicate with a database server***.
  - application program interface (API) to: ***open a connection with a database; send queries and updates; get back results***.
- Applications such as GUI, spreadsheets, etc. can use ODBC
- Defined originally for Basic and C, versions available for many languages.
- Each database system supporting ODBC provides a **"driver" library** that must be linked with the client program.
- When client program makes an ODBC API call, the code in the library ***communicates with the server to carry out the requested action, and fetch results***.

Embedded SQL

- The SQL standard defines **embeddings of SQL** in a variety of programming languages such as C, Java, and Cobol.
- A language to which SQL queries are embedded is referred to as a **host language**, and the SQL structures permitted in the host language comprise ***embedded* SQL**.
- An embedded SQL program must be processed by a ***special preprocessor*** prior to compilation (replacing embedded SQL requests with host-language declarations and procedure calls that allow runtime execution of the database accesses; the resulting program is compiled by the host-language compiler).
- In JDBC, SQL statements are ***interpreted at runtime***. When embedded SQL is used, some SQL-related errors (including data-type errors) may be caught at compile time.

Procedural Extensions and Stored Procedures

- SQL provides a **module language**: ***permitting definition of procedures in SQL, with if-then-else statements, for and while loops, etc***.
- **Stored Procedures**
  - Can ***store procedures*** in the database
  - Execute them using the ***call statement***
  - Permit ***external applications to operate on the database*** without knowing about internal details

SQL Functions

- Define a function that, given the *name* of a *department*, returns the count of the number of *instructors* in that *department*.

```
create function dept_count (dept_name varchar(20))
returns integer
begin
    declare d_count integer;
    select count (* ) into d_count
    from instructor
    where instructor.dept_name = dept_name
    return d_count;
end
```

- Find the *department name* and *budget* of all *departments* with more than 12 *instructors*.

```
select dept_name, budget
from department
where dept_count (dept_name ) > 12
```

Table Functions

- Given the *name* of a *department*, return all *instructors*

```
create function instructors_of (dept_name char(20))
        returns table (  ID varchar(5),
                         name varchar(20),
                         dept_name varchar(20),
                         salary numeric(8,2))
return table
        (select ID, name, dept_name, salary
         from instructor
         where instructor.dept_name = instructors_of.dept_name)
```

- Usage

```
select *
from table (instructors_of ('Music'))
```

SQL Procedures

- The *dept_count* function could instead be written as procedure:

```
create procedure dept_count_proc (in dept_name varchar(20),
                                  out d_count integer)
begin
    select count(*) into d_count
    from instructor
    where instructor.dept_name = dept_count_proc.dept_name
end
```

- Procedures can be invoked either from an ***SQL procedure*** or from ***embedded SQL***, using the **call** statement.

```
declare d_count integer;
call dept_count_proc( 'Physics', d_count);
```

Procedural Constructs

- **Variable declaration**: **declare** ; **Assignment**: **set**
- **Compound statement**: **begin** … **end**
  - May contain multiple SQL statements between **begin** and **end**.
  - Local variables can be declared within a compound statements
- **While** and **repeat** statements:

```
declare n integer default 0;
while n < 10 do
    set n = n + 1
end while

  repeat
  set n = n  – 1
  until n = 0
end repeat
```

Procedural Constructs (Cont.)

- ***For*** loop
  - Permits iteration over all results of a query
  - Example:

```
declare n integer default 0;
for r as
        select budget from department
        where dept_name = 'Music'
do
        set n = n + r.budget
end for
```

  - The statement ***leave*** can be used to exit the loop;
  - While ***iterate*** starts on the next tuple, from the beginning of the loop, skipping the remaining statements.

Procedural Constructs (Cont.)

- **Conditional statements** (**if-then-else**)

```
if boolean expression
        then statement or compound statement
elseif boolean expression
        then statement or compound statement
else statement or compound statement
end if
```

- SQL:1999 also supports a **case** statement similar to C case statement

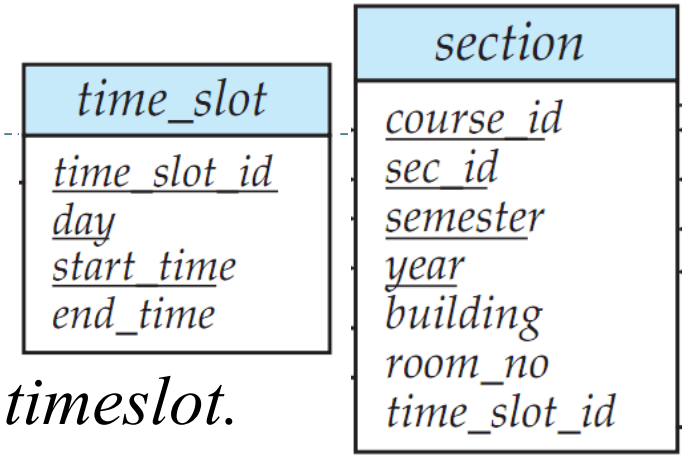
Triggers

- A **trigger** is a statement that is ***executed automatically by the system as a side effect of a modification to the database***.
- To design a **trigger mechanism**, we must:
  - ***Specify the conditions under which the trigger is to be executed.***
  - ***Specify the actions to be taken when the trigger executes.***
- Triggers introduced to SQL standard in SQL:1999, but supported even earlier using non-standard syntax by most databases.
  - **Syntax illustrated here may not work exactly on your database system; check the system manuals**

Need for triggers

- Triggers can be used to ***implement certain integrity constraints*** that cannot be specified using the constraint mechanism of SQL.
  - Whenever a tuple is inserted into the *takes* relation, updates the tuple in the *student* relation for the student taking the course by adding the number of credits for the course to the student's total credits.
- Triggers are also useful mechanisms for ***alerting humans*** or for ***starting certain tasks automatically*** when certain conditions are met.
  - Suppose a warehouse wishes to maintain a minimum inventory of each item; when the inventory level of an item falls below the minimum level, an order can be placed automatically.

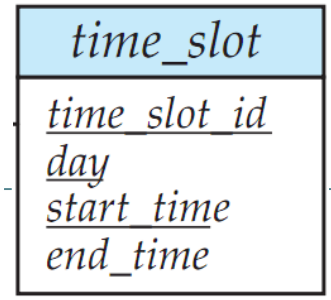
Trigger Example

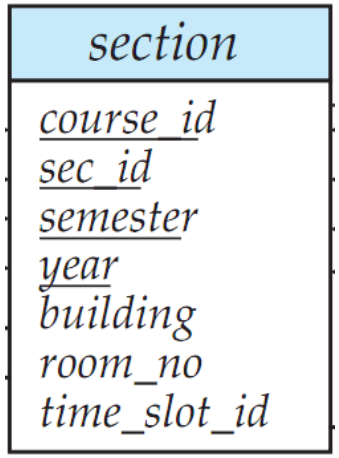


- E.g. *time_slot_id* is **not** a primary key of *timeslot,* so we **cannot** create a foreign key constraint from *section* to *timeslot.*
- Alternative: ***use triggers on section and timeslot to enforce integrity constraints***

```
create trigger timeslot_check1 after insert on section
referencing new row as nrow
for each row
when (nrow.time_slot_id not in (
            select time_slot_id
            from time_slot)) /* time_slot_id not present in time_slot */
begin
   rollback
end;
```

Triggers in SQL

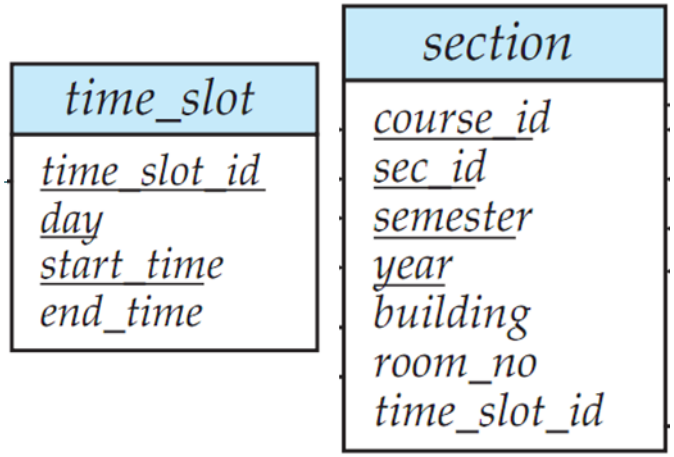





```
create trigger timeslot_check1 after insert on section
referencing new row as nrow
for each row
when (nrow.time_slot_id not in (
            select time_slot_id
            from time_slot)) /* time_slot_id not present in time_slot */
begin
   rollback
end;
```

- An SQL **insert** statement could insert multiple tuples of the relation, and the ***for each row*** clause in the trigger code would then explicitly iterate over each inserted row.
- The **referencing new row as** clause creates a variable ***nrow*** (called a ***transition variable***) that stores the value of an inserted row after the insertion.
- The **when** statement specifies a condition.

Trigger Example (Cont.)




```
create trigger timeslot_check2 after delete on timeslot
referencing old row as orow
for each row
when (orow.time_slot_id not in (
            select time_slot_id
            from time_slot) /* last tuple for time_slot_id deleted from time_slot */
    and orow.time_slot_id in (
            select time_slot_id
            from section)) /* and time_slot_id still referenced from section*/
begin
    rollback
end;
```

- The **referencing old row as** clause can be used to create a variable storing the old value of an updated or deleted row.
- This trigger checks that the *time_slot_id* of the tuple being deleted is either **still present in *time_slot*** , or that **no tuple in *section* contains that particular *time_slot_id* value**; otherwise, referential integrity would be violated.

Triggering Events and Actions in SQL

- **Triggering event** can be **insert**, **delete** or **update**
- Triggers on update can be restricted to specific attributes
  - **E.g., after update of** *takes* **on** *grade*
- Values of attributes before and after an update can be referenced
  - **referencing old row as**: for deletes and updates
  - **referencing new row as**: for inserts and updates
- Triggers can be activated ***before an event***, which can serve as extra constraints. E.g. convert blank grades to *null*.

```
create trigger setnull before update on takes
referencing new row as nrow
for each row
when (nrow.grade = ' ')
begin atomic
    set nrow.grade = null;
end;
```

Trigger to Maintain *credits_earned* value

- A trigger can be used to **keep the *tot_cred* attribute value of *student* tuples up-to-date** when the *grade* attribute is updated for a tuple in the *takes* relation.

```
create trigger credits_earned after update of takes on (grade)
referencing new row as nrow
referencing old row as orow
for each row
when nrow.grade <> 'F' and nrow.grade is not null
    and (orow.grade = 'F' or orow.grade is null)
begin atomic
    update student
    set tot_cred= tot_cred+
            (select credits
             from course
             where course.course_id= nrow.course_id)
    where student.id = nrow.id;
end;
```

When Not To Use Triggers

- Triggers were used earlier for **tasks** such as
  - ***maintaining summary data*** (e.g., total *salary* of each *department*)
  - ***replicating databases*** by recording changes to special relations (called ***change*** or ***delta*** relations) and having a separate process that applies the changes over to a replica
- There are better ways of doing these now
  - Databases today provide ***built in*** **materialized view** facilities to maintain summary data
  - Databases provide ***built-in*** **support for replication**

When Not To Use Triggers

- **Risk of unintended execution of triggers** (when loading data from a backup copy or replicating updates at a remote site): the triggered action has already been executed, and typically ***should not be executed again***.
- Other risks with triggers:
  - **Trigger error** detected at runtime leading to failure of critical transactions that set off the trigger
  - **Cascading execution**
- Triggers can serve a very useful purpose, but they are ***best avoided when alternatives exist***.

Homework

- 5.8, 5.15

5.8 Consider the bank database of Figure 5.25. Write an SQL trigger to carry out the following action: On **delete** of an account, for each owner of the account, check if the owner has any remaining accounts, and if she does not, delete her from the *depositor* relation.

branch(*branch_name*, *branch_city*, *assets*)
customer (*customer_name*, *customer_street*, *cust omer_city*)
loan (*loan_number*, *branch_name*, *amount*)
borrower (*customer_name*, *loan_number*)
account (*account_number*, *branch_name*, *balance*)
depositor (*customer_name*, *account_number*)

Figure 5.25 Banking database for Exercises 5.7, 5.8, and 5.28 .

Homework

5.15 Consider an employee database with two relations

employee (*employee_name*, *street*, *city*)
works (*employee_name*, *company_name*, *salary*)

where the primary keys are underlined. Write a query to find companies whose employees earn a higher salary, on average, than the average salary at "First Bank Corporation".

a. Using SQL functions as appropriate.

b. Without using SQL functions.